CS & IT ENGINEERING



Algorithms

Analysis of Algorithms



Lecture No.- 09

By- Aditya sir

Recap of Previous Lecture



- Topic
- Topic

Questions

Analysis of an Algo

Topics to be Covered



- Topic
- Topic
- Topic

Recursive Algo

Practice Questions



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Analysis of Algorithms

#Q. An element in an Array is called Leader if it is greater than all elements to the right of it. The time complexity of the most efficient algorithm to print all Leaders of the given Array of size 'n' is_____.



Topic : Analysis of Algorithms

Summary

	Best Case	Worst Case
1. Brute force	$O(n) = \Omega(n)$	$O(n^2)$
2. Efficient Algo	$O(n) = \Omega(n)$	$O(n)$

W.C

Efficient



Topic : Analysis of Algorithms

#Q. Arrange the following function in increasing order of their growth rate?

$f_1 \to nlogn$, $f_2 \to n^2$, $f_3 \to e^n$, $f_4 \to n^{3/2}$, $f_5 \to n$, $f_6 \to (1/n)$, $f_7 \to 2^n$

P P E P P D E

A $F_2, F_1, F_4, F_3, F_5, F_7, F_6$

B $F_6, F_5, F_1, F_4, F_2, F_3, F_7$

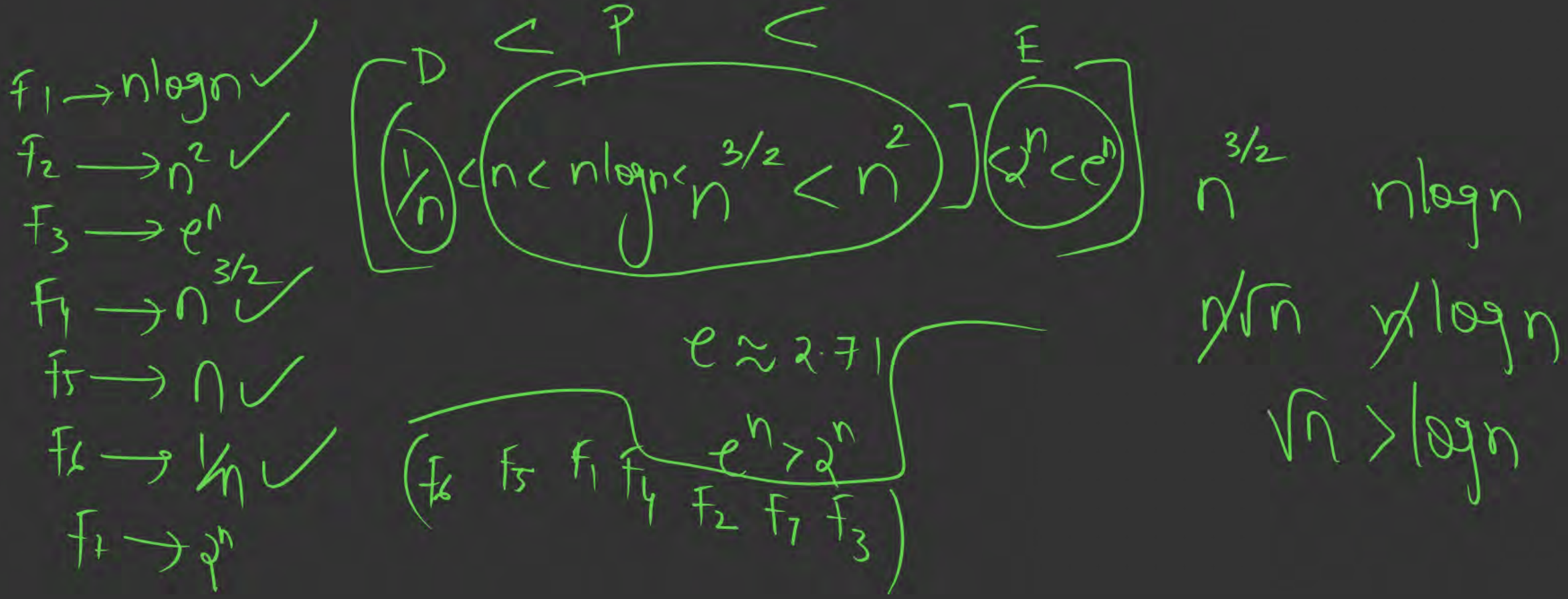
C $F_6, F_5, F_1, F_4, F_2, F_7, F_3$

D $F_6, F_5, F_1, F_4, F_7, F_3, F_2$

$e > 2$

$e^n > 2^n$

$f_3 > f_7$


$f_1 \to n\log n$
$f_2 \to n^2$
$f_3 \to e^n$
$f_4 \to n^{3/2}$
$f_5 \to n$
$f_6 \to 1/n$
$f_7 \to 2^n$
D
P
E
$\frac{1}{n} < n < n\log n < n^{3/2} < n^2$
$2^n < e^n$
$e \approx 2.71$
$e^n > 2^n$
$(f_6\ f_5\ f_1\ f_4\ f_2\ f_7\ f_3)$
$n^{3/2}$ $n\log n$
$n/\sqrt{n}$ $n/\log n$
$\sqrt{n} > \log n$



Topic : Analysis of Algorithms

#Q. Arrange in decreasing order of growth.

$f_1 \rightarrow n^{1/3}$, $n(\log n)^{10}$, $n^{7/4}$, $(2.002)^n$, $f_5 \rightarrow e^n$

A $F_5 > F_4 > F_1 > F_3 > F_2$

B $F_4 > F_5 > F_2 > F_3 > F_1$

C $F_4 > F_5 > F_3 > F_2 > F_1$

D $F_5 > F_4 > F_3 > F_2 > F_1$

f_1, f_2, f_3, f_4, f_5

$f_1 = n^{1/3} \longrightarrow P$

$f_2 = n * (\log n)^{10} \longrightarrow P$

$f_3 = n^{7/4} \longrightarrow P$

$f_4 = (2.002)^n \longrightarrow E$

$f_5 = e^n \longrightarrow E$

$(f_1 < f_2 < f_3 < f_4 < f_5)$

$n^{1/3} < n(\log n)^{10}$

$n^{1/3} \quad n^{7/4}$

$n^{1/3} \quad n^{1+3/4}$

$n^{1/3} < n \times n^{3/4}$

$n(\log n)^{10} < n \times n^{3/4}$

$$(2.002)^n < e^n$$

$$2.002 < 2.71$$

$$(f_4 < \bar{f}_5)$$



Topic : Analysis of Algorithms

Frame work to determine the time complexity of recursing algorithm.



Topic : Analysis of Algorithms

Recursion:

$n! = n * (n-1) * (n-2) \ldots\ldots 1$

$5! = f(5) = 5 * f(4)$

$4 * f(3)$

$3 * f(2)$

$2 * f(1)$

$f(n) = n * f(n-1)$

$2 * 1 = 2$

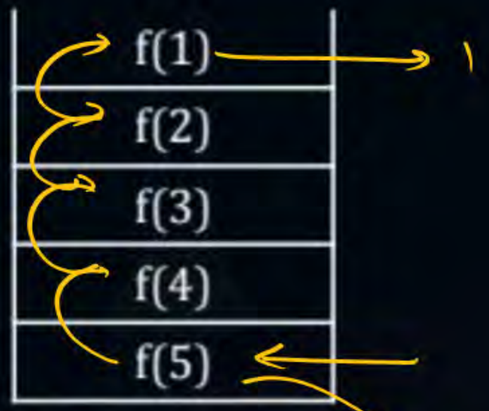
$3 * 2 = 6$

$4 * 6 = 24$

$5 * 24 = 120$

Base condition terminating condition $f(1) = 1$

Recursion stack



$f(5) = 5! = 120$

120



Topic : Analysis of Algorithms

Framework to determine TC of recursive algo.

3 Steps:-

Step-1. Find (Recursive equation for TC) for the recursive code.

→ TC Recurrence

Step-2. Solve this Recursive equation to get mathematical expression/value

Step-3. Apply Asymptotic notation on this value.

→ 3 mtds

(Asymptotic Analysis of Recursive Algo)



Topic : Analysis of Algorithms

TC Recursive equation

1. Back substitution method

- General approach
- Gives
  - Values of recurrence
  - Also give TC→ O,O,Ω

2. Master Method/ Theorem

Work in specific case

- Only gives final TC but not value

→ Shortcut

3. Recursive Tree (Approx.)

TC(Final but not value)



Topic : Analysis of Algorithms

Example:-

```
Algo f(n)
{
    if (n==1)
        return 1
    return n*f(n-1)
}
```

T(n)

C1

T(n-1)

$F(n) = n * F(n-1)$

$n * F(n-1)$

$\downarrow$

$t(n-1)$



Topic : Analysis of Algorithms

Let T(n) Represent the time complexity of f(n)

Step1:

$$T(n) = T(n-1) + C \quad n \geq 1$$

$$T(n) = C1 \ , n = 1$$

$$T(n) = C1 \ , \ n = 1$$

$$T(n) = T(n-1) + C \ , \ n > 1$$

Step 2:

$T(n) = T(n-1) + C$ — (1)

$T(n-1) = T(n-2) + C$

$T(n) = (T(n-2) + C) + C$

$T(n) = T(n-2) + 2C$ — (2)

$T(n-2) = T(n-3) + C$

$T(n) = T(n-3) + C + 2C$

$T(n) = T(n-3) + 3C$ — (3)

$T(n) = T(n-4) + 4C$ — (4)

In general

$T(n) = T(n-K) + K * C$ — (5)

For Base C, $n - K = 1$

$K = n - 1$

$$T(n) = T(n-k) + k * C$$

Put $k = n-1$

$$T(n) = T(1) + (n-1) * C$$

$$T(n) = C1 + (n-1) * C$$

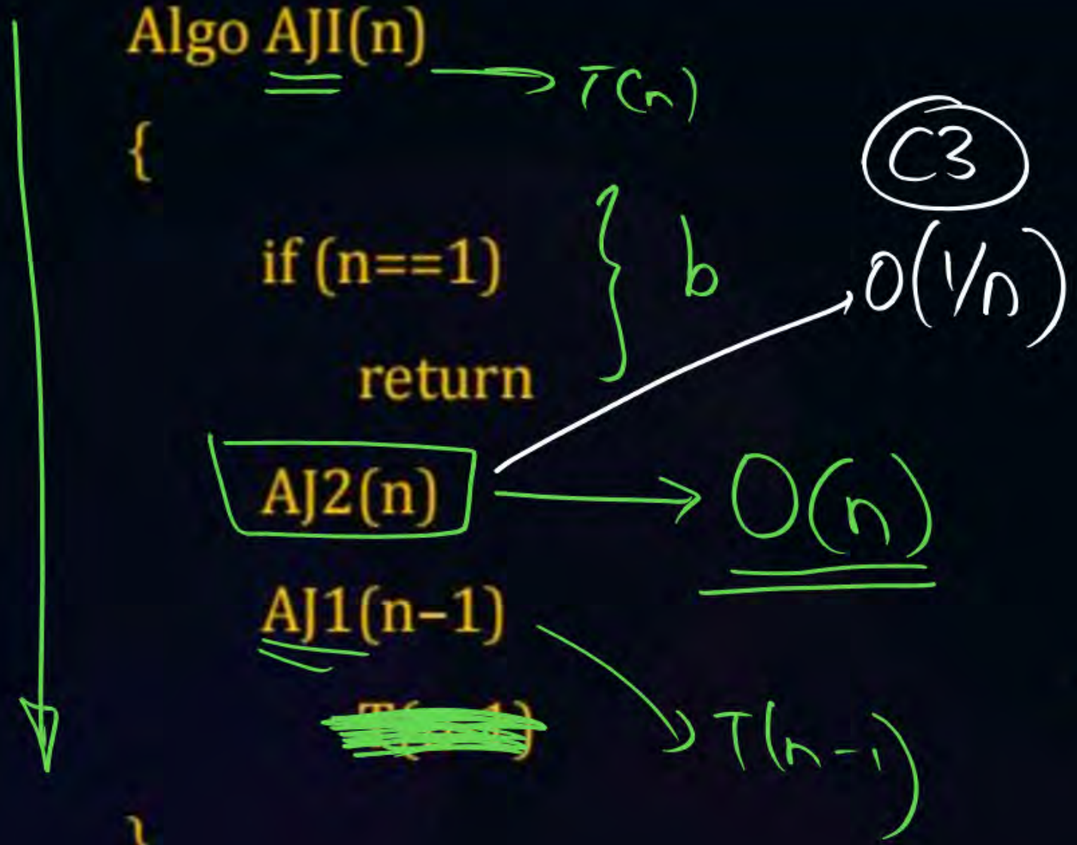
★ value of Reurrence

Step 3 :-

$$T(n) = O(n)$$

Topic : Analysis of Algorithms

Eg.1.



$O(n) = b \times n$

Case 2.

$T(n) = b, \quad n=1$

$T(n) = a + O(n) + T(n-1)$

$\star$ $T(n) = a + n + T(n-1)$



Topic : Analysis of Algorithms

Case-1:

Assume AJ2 (n) takes constant time → O(1)

$$T(n) = T(n-1) + C, \quad n > 1$$

$$T(n) = C1, \quad n = 1$$



Topic : Analysis of Algorithms

Case-2:

Assume AJ2 (n) takes constant time → O(n)

$$T(n) = b, \quad n = 1$$

$$T(n) = T(n-1) + n + a, \quad n > 1$$

$$T(n) = T(n-1) + n + a \quad \text{——} (1)$$

$$T(n-1) = T(n-2) + (n-1) + a$$

$$T(n) = \left[T(n-2) + (n-1) + a\right] + n + a$$

$$T(n) = T(n-2) + n + (n-1) + 2a \quad \text{——} (2)$$

$$T(n-2) = T(n-3) + (n-2) + a$$

$$T(n) = \left(T(n-3) + (n-2) + a\right) + n + (n-1) + 2a$$

$$T(n) = T(n-3) + \left[n + (n-1) + (n-2)\right] + 3a \quad \text{——} (3)$$

In general

$$T(n) = T(n-k) + \left[n + (n-1) + (n-2) \cdots (n-(k-1))\right] + k * a \quad \text{— (4)}$$

for B.C, $n - k = 1 \Rightarrow k = (n-1)$

$$T(n) = T(1) + \left[n + (n-1) \cdots + (n-(n-1-1))\right] + (n-1) * a$$

$$= b + \left[n + (n-1) \cdots + (n-n+2)\right] + (n-1) * a$$

$$= b + \left[n + (n-1) + \cdots 2\right] + (n-1) * a$$

$$T(n) = b + \left(\sum_{i=1}^{n} i - 1 \right) + (n-1) * a$$

$$T(n) = b + \left(\frac{n(n+1)}{2} - 1 \right) + (n-1) * a \longrightarrow \text{value}$$

Step3 :- $\boxed{T(n) = O(n^2)}$



Topic : Analysis of Algorithms

Case-3:

Assume AJ2 (n) takes constant time → O(1/n)

$T(n) = b, n = 1$

$T(n) = T(n-1) + \frac{1}{n} + a, \quad n > 1$

$$T(n) = T(n-1) + \frac{1}{n} + a \quad \text{— (1)}$$

$$T(n-1) = T(n-2) + \frac{1}{(n-1)} + a$$

$$T(n) = T(n-2) + \frac{1}{(n-1)} + a + \frac{1}{n} + a$$

$$T(n) = T(n-2) + \left[\frac{1}{n} + \frac{1}{n-1}\right] + 2a \quad \text{— (2)}$$

$$T(n-2) = T(n-3) + \frac{1}{(n-2)} + a$$

$$T(n) = \left(T(n-3) + \frac{1}{(n-2)} + a\right) + \left[\frac{1}{n} + \frac{1}{(n-1)}\right] + 2a$$

$$T(n) = T(n-3) + \left[\frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2}\right] + 3a \quad \text{——} \quad (3)$$

In general, $T(n) = T(n-k) + \left[\frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} \cdots + \frac{1}{(n-(k-1))}\right] + k * a$

for B.C, $n-k=1 \longrightarrow k=(n-1)$

$$T(n) = T(n-k) + \left[\frac{1}{n} + \frac{1}{n-1} \cdots + \frac{1}{2}\right] + (n-1) * a$$

$\longrightarrow (\log n - 1)$

$$T(n) = T(1) + (\log n - 1) + (n-1) * a$$

$$T(n) = b + (\log n - 1) + na - a$$ → value of Rec

$$T(n) = O(n)$$



Topic : Analysis of Algorithms

2. Algo AJ(n)

```
{
    if (n ==1)
        return 1
    else
        return ((AJ(n-1) + AJ (n-1))
}
```

T(n-1) T(n-1)

O(n)
O(n+n) = O(n) ✗

$$T(n) = b, \quad n = 1$$

$$T(n) = T(n-1) + T(n-1) + a, \quad n > 1$$

$$T(n) = 2T(n-1) + a, \quad n > 1$$

$T(n) = 2T(n-1) + a$

$T(n-1) = 2T(n-2) + a$

$T(n) = 2[2T(n-2) + a] + a$

$= 2^2 T(n-2) + 2a + a$

$T(n) = 2^2 T(n-2) + 3a$ — (2)

$T(n-2) = 2T(n-3) + a$

$T(n) = 2^2[2T(n-3) + a] + 3a$

$= 2^3 T(n-3) + 4a + 3a$

$\hookrightarrow (4-1)a$

$T(n) = 2^3 T(n-3) + 7a$ — (3)

$\hookrightarrow (8-1)a$

In general, $T(n) = 2^k T(n-k) + (2^k - 1) * a$ — (4)

$$n - k = 1$$

$$k = (n-1)$$

$$T(n) = 2^{(n-1)} * T(1) + (2^{n-1} - 1) * a$$

$$T(n) = 2^{n-1} * b + (2^{n-1} - 1) * a \longrightarrow \text{value}$$

$$T(n) = O(2^{n-1})$$

$$\boxed{T(n) = O(2^n)}$$



Topic : Analysis of Algorithms

1. Algo AJ(n)

```
{
    if (n ==2)
        return 2
    else
        return AJ(√n)
}
```

H.W



Summary

Recursive Algo TC



THANK - YOU